Week 08 Move Ordering Optimization

Project: Research Design and Implementation of a Gomoku Game Engine with Artificial Intelligence
Reporting period: Week 08
Report date: 17 October 2026
Status: Existing reference ordering implemented and evaluated with local controlled comparisons
Evidence scope: Native Rust reference core and identical candidate search trees

1 Abstract

Move ordering changes the sequence in which search explores moves, helping alpha beta obtain useful bounds earlier. This week reviews the actual scoring policy in caro-core-reference and compares ordering disabled and enabled on identical candidate trees.

The implementation scores temporary child states, restores them, and sorts by decreasing score. It uses root perspective at maximizing and minimizing nodes, with separate immediate win priority. This differs from the earlier description of ascending scores at minimizing nodes. A permutation preserves the completed minimax value, but its scoring overhead can exceed recursive savings.

2 Research Questions and Scope

RQ 08 1 asks whether the completed root score agrees when only ordering changes. RQ 08 2 asks how ordering changes nodes, cutoffs, skipped siblings, and preliminary scoring work. RQ 08 3 asks whether a reduction in recursive work produces a local elapsed time improvement. RQ 08 4 asks how evaluation perspective and equal scores affect interpretation of the selected move.

[A] denotes inspected source and current execution; [B] denotes a primary research publication; [C] denotes derivation or inference; [D] denotes a recommendation. This report studies a reference search implementation. The reported durations do not establish browser response time, Wasm speed, or deployed engine behavior. Transposition table and iterative deepening effects are reserved for Week 09, so they are not silently mixed into this ablation.

3 Source Evidence and Ownership

File	Responsibility	Evidence and assessment
caro-core-reference/src/search/ordering.rs	Preliminary scoring and permutation	[A] order_moves uses play and undo, evaluates from root, and sorts descending score with ascending index ties. High confidence.
caro-core-reference/src/search/alpha_beta.rs	Bound updates and recursive traversal	[A] value chooses maximum or minimum from state.to_move compared with root; ordering is called before the child loop. High confidence.
caro-core-reference/src/evaluator.rs	Root relative static value	[A] evaluate accepts a perspective player, returns terminal mate signs, and scores attack minus weighted defense. High confidence.
caro-core-reference/src/state.rs	Reversible position changes	[A] play returns an UndoToken; undo restores board, mover, count, status, and last move. High confidence.

File	Responsibility	Evidence and assessment
caro-core-reference/ src/metrics.rs	Search instrumentation	[A] SearchStats separates nodes_visited, leaf_evaluations, ordering_evaluations, cutoffs, and skipped_siblings. High confidence.

The state owner is GameState. The ordering function borrows mutable state only to score hypothetical children and must restore each transition before proceeding. The search owns alpha and beta, not the ordering function. The scoring function owns numerical evaluation, while candidate generation owns membership. This direction permits ordering to be disabled without changing the search configuration's action set.

[A] The earlier report claimed an ascending comparator at minimizing nodes. order_moves contains no such branch. This source discrepancy is established with high confidence; its performance implications require measurement.

4 Mathematical Model and Value Invariance

Let $M(s)$ be the fixed generated move list and π_s a permutation of it. With root player p, depth limited minimax is

$$V(s,d)=\begin{cases} T(s,p), & s \text{ is terminal}, \\ E(s,p), & d=0, \\ max_{m\in M(s)} V(s_m, d-1), & \text{mover}=p, \\ min_{m\in M(s)} V(s_m, d-1), & \text{mover}\neq p. \end{cases}$$

[C] A maximum or minimum over a finite collection is invariant under permutation. Induction on remaining depth therefore proves value equality when every node retains the same children, terminal rules, evaluator, and depth. At depth zero both policies return the same evaluation. At positive depth the induction hypothesis gives equal child values, and permuting those values preserves their maximum or minimum. Correct alpha beta preserves that minimax value while avoiding branches whose bounds show they cannot change the result.

This argument requires a completed search. A budgeted or interrupted search can have different information available after the same amount of work because it has visited a different prefix. It also requires the same candidate policy. Sorting a capped list and scoring an uncapped list before truncation are different operations: the latter changes which moves remain and therefore exceeds pure ordering.

[B] Knuth and Moore analyze alpha beta correctness and the relationship between traversal and search effort. In an ideal uniform tree with perfect ordering, the minimum leaf count has the familiar form below, yielding best case growth near $b^{d/2}$ for even depth. Worst case traversal can still approach b^d. These are theoretical models rather than measured performance guarantees. Knuth and Moore 1975.

$$L_{ideal}(b,d)=b^{\lceil d/2\rceil}+b^{\lfloor d/2\rfloor}-1.$$

5 Actual Preliminary Scoring Policy

For each candidate m, the implementation computes a priority using the state after the current mover plays it. A preferred move, when supplied, receives the maximum integer priority without the ordinary preliminary scoring step. Otherwise an immediate win by the mover receives MATE_SCORE. All remaining successful plays use evaluate with the root player as perspective. The basic alpha beta comparisons in this week pass no preferred move.

```
OrderMoves state moves root preferred
  scored = empty list
  for each move m
    if m is preferred set score to maximum integer
    else
      token = play m
      increment ordering evaluations
      score = mover win priority or evaluate from root
      undo token
    append m and score
  sort by descending score then ascending index
  copy moves back
```

The source also handles a failed play by assigning the minimum integer score without incrementing the scoring counter. Generated legal lists should avoid that path. This pseudocode focuses on successful generated moves and leaves the source as the authoritative error handling contract.

[C] For a root relative heuristic, an intuitively promising maximizing child has a high score and a promising minimizing child has a low score. Thus descending order at maximizing nodes and ascending order at minimizing nodes would align the comparator with that perspective. The actual implementation does the first at both kinds of node. Its separate immediate mover win priority means opponent wins can still be promoted, but it does not generally reverse every ordinary heuristic score at minimizing nodes.

[D] A future comparator change should be evaluated as a separate experiment: preserve membership, reverse only the nonterminal root relative ordering where appropriate, and compare score equivalence and total cost. This report records the existing behavior; it does not claim that the suggested comparator has been implemented or benchmarked.

6 Ties and Move Identity

[A] At the root, search updates the selected move only when the new score is strictly greater than the current best, apart from initialization. Therefore the first encountered move among equal best values is retained. Without ordering this is ordinarily the earliest row major candidate. With ordering it is the earliest candidate after preliminary scores have been sorted, with coordinate index resolving equal preliminary scores.

[C] Equal final minimax values do not imply equal preliminary heuristic scores. Two winning choices may both have final value v, while one has a larger one step score and is searched first under ordering. The selected coordinate can change while both choices remain optimal in the declared tree. Consequently completed root score equality is the primary semantic check. Exact coordinate equality is a useful observed result when it occurs, but it is too strong as a universal requirement unless a shared canonical tie rule is explicitly enforced.

7 Worked Bound Example

Consider a maximizing root with three minimizing children. Their leaf values are A with 4, 5, 6; B with 3, 9, 8; and C with 2, 7, 6. Their exact minimum values are 4, 3, and 2, so the root value is

$$max(min(4,5,6),min(3,9,8),min(2,7,6))=4.$$

[C] After searching A completely, alpha is 4. At B, seeing 3 first establishes that B cannot beat the root's current choice, so its remaining two leaves can be skipped. Seeing 2 first at C likewise permits a cutoff. Only five leaves are needed: three for A and one each for B and C. If B is visited as 9, 8, 3 and C as 7, 6, 2, both require three leaves before the same bound is established, for nine leaves overall.

This is an illustrative mathematical tree, not a measured Gomoku fixture. It explains why low root relative scores can be useful first at minimizing nodes and why a cutoff count alone does not identify ordering quality. The timing benefit depends on the work spent constructing that sequence.

8 Cost Model and Instrumentation

Let c_v be the candidate count at internal node v, and let H_v be the cost of playing, preliminary scoring, and undoing one candidate there. Ordering adds approximately

$$Q=\sum_v c_v, T_{order} \approx \sum_v \left(c_v H_v + O\left(c_v \log c_v\right)\right).$$

[C] Recursive savings are beneficial only when the avoided subtree work exceeds this additional cost. At depth one, ordering cannot skip ordinary root children under a full initial window, so it adds preliminary work without reducing the number of visited leaves. At greater depths its benefit depends on bound quality, branching, and the evaluator's cost.

[A] nodes_visited counts recursive search entries, including root and terminal or depth limited leaves. ordering_evaluations counts successfully played nonpreferred scoring attempts, including those using the terminal win priority. It is not an exact count of calls to the full nonterminal evaluator. leaf_evaluations also includes terminal score returns. Their sum is a useful count of scoring events, but not an instruction count.

[A] skipped_siblings counts unvisited immediate siblings at cutoff points. It does not count all their hypothetical descendants. The current metrics do not expose the child index of every cutoff, so a first child cutoff percentage cannot be recovered from these counters. No first move cutoff ratio is claimed in this report.

9 Controlled Experimental Method

The harness in research/week_05_09 uses a separate native research package linked to the reference core. Each ordering pair starts from the same constructed position and uses identical rule configuration, evaluator weights, depth, radius, and cap. Only use_move_ordering changes. Root state is compared before and after searching so that performance results are not accepted while silently leaving a hypothetical stone in the position.

A capped comparison can isolate ordering because both policies receive the same cap before the ordering switch. It cannot validate the uncapped default or establish tactical safety of that cap. The protocol also includes an uncapped comparison at a bounded depth. Runs report recursive and preliminary scoring counters independently, plus local elapsed duration. Fixed fixture repetitions support repeatability within this host; they are not independent player trials or a population estimate of game strength.

10 Experimental Results

[A] results/verification.json records 112 ordering score comparisons passing and 551 root restoration checks across the wider experiment. Controlled cases below use three repeats. N denotes natural order, O the actual heuristic policy, and Q ordering_evaluations. Units are microseconds; ranges retain observed timing variation.

Setting	Policy	Nodes	Q	Median	Range
Cluster full depth 2	N	10939	0	16442	13561 to 19989
Cluster full depth 2	O	2164	48841	68457	35516 to 73064
Cluster radius 1 depth 2	N	64	0	41	41 to 47
Cluster radius 1 depth 2	O	102	188	196	195 to 207
Win radius 1 depth 2	N	237	0	167	166 to 170

Setting	Policy	Nodes	Q	Median	Range
Win radius 1 depth 2	O	39	416	326	320 to 331
Cluster cap 12 depth 3	N	402	0	278	278 to 280
Cluster cap 12 depth 3	O	507	1008	1062	1055 to 1102

Radius one rows are uncapped unless cap 12 is explicit. The shared scores are respectively 0, 0, 9999999, and 4910. Cluster full reduces recursive nodes by 80.22 percent, yet its median time increases about 4.16 times. The win case saves nodes while preliminary scoring makes the median slower. Ordering is therefore not demonstrated as a net speed improvement here.

[A] In the block fixture at depth two over all legal moves, natural order selects $(0,0)$ and ordering selects $(7,2)$ with the same score −9999998. The opponent has two winning endpoints; this is an observed equal value coordinate change, not evidence of a successful defense.

Reproduce with python3 research/week_05_09/run.py. results/search_runs.csv retains raw trials; search_summary.csv gives medians and ranges. provenance.json records Rust 1.98.1 release, Linux x86 64, Intel Xeon W 11855M, and source hashes. Only the search call is timed; host load is uncontrolled.

11 Discussion and Validity Limits

[C] An optimization must improve a relevant resource measure while preserving the completed value. The recorded node reductions often coexist with increased preliminary scoring and elapsed time. Reporting both prevents a misleading efficiency conclusion.

The small fixture corpus cannot determine the best policy for every opening or endgame. The contiguous run evaluator omits full broken threat semantics. Native measurements exclude worker messaging, Wasm initialization, UI rendering, and device scheduling.

The old fixed speedup and first move cutoff claims lack supporting trials. They are replaced here by the recorded comparisons and explicit instrumentation limits.

12 Traceability and Next Steps

Question	Evidence	Bounded conclusion
RQ 08 1	Same tree comparisons and permutation proof	Completed value should be preserved; observed agreement is fixture scoped.
RQ 08 2	SearchStats and evidence tables	Recursive work and scoring overhead must be considered together.
RQ 08 3	Local elapsed trials	Speed conclusions apply to this native host and configuration.
RQ 08 4	order_moves and strict root comparison	Perspective affects efficiency and ties can alter coordinates.

[D] Retain separate counters for search and preliminary work. Evaluate perspective aware ordering as an isolated follow up before enabling it as a product default. If stable coordinate identity matters to visualization or tests, specify a canonical final value tie rule rather than assuming traversal order already supplies one. Week 09 can then study preferred moves and repeated searches without confusing their benefits with this week's standalone ordering policy.

13 References

1. [A] caro-core-reference/src/search/ordering.rs, order_moves, inspected 17 October 2026.
2. [A] caro-core-reference/src/search/alpha_beta.rs and src/search/minimax.rs, root selection and traversal, inspected 17 October 2026.
3. [A] caro-core-reference/src/evaluator.rs, evaluate and MATE_SCORE, and src/metrics.rs, SearchStats, inspected 17 October 2026.
4. [A] research/week_05_09, controlled native harness and accompanying raw evidence.
5. [B] Knuth D E and Moore R W. An Analysis of Alpha Beta Pruning. Artificial Intelligence 6, 1975, pages 293 to 326. Publisher record and primary paper PDF.